

Final Exam — Solutions

Software Engineering — UATX — Spring 2026

Question 1

This component renders a shopping cart. Work out what it puts on the screen for an empty cart (`items` is `[]`) and for a cart with two products. One of those cases shows something the developer did not intend.

```
function Cart({ items }) {  
  return (  
    <div>  
      <h2>Your cart</h2>  
      {items.length && (  
        <ul>  
          {items.map(i => <li key={i.id}>{i.name}</li>)}  
        </ul>  
      )}  
      {items.length > 0 && <button>Check out</button>}  
    </div>  
  );  
}
```

1. What is it?
2. Why does it happen?
3. How could you fix it?

SOLUTION

1. On the empty cart it renders the `<h2>Your cart</h2>`, then a stray `0` as text on the page (and no button). The two-product cart renders as intended -- the list and the "Check out" button. The unintended thing is that stray `0`.
2. `items.length` is `0` on an empty cart, and `0 && <ul>...` evaluates to `0` -- JS returns the falsy left operand -- and React renders the number `0` as text. (With two items, `2 && <ul>...` returns the `<ul>`, so it renders fine.) The second line, `items.length > 0 && <button>`, is already a boolean (`false` on an empty cart), and React renders nothing for `false`, which is why the button is correctly absent.
3. Make the left side a real boolean so it can never be the number `0`: `items.length > 0 && <ul>...` (or `items.length ? <ul>... : null`).

Question 2

`products.review_count` is a denormalized counter kept in sync as reviews are added. Two shoppers review the same product at the same instant; `review_count` starts at 5. Both transactions run their `SELECT review_count` before either runs its `UPDATE`.

```
@app.post("/api/products/{product_id}/reviews")
def add_review(product_id: int, body: ReviewIn,
               user: User = Depends(current_user)):
    with engine.begin() as conn:
        conn.execute(
            text("INSERT INTO reviews (product_id, author_id, body) "
                  "VALUES (:p, :u, :b)"),
            {"p": product_id, "u": user.id, "b": body.text},
        )
        row = conn.execute(
            text("SELECT review_count FROM products WHERE id = :p"),
            {"p": product_id},
        ).first()
        conn.execute(
            text("UPDATE products SET review_count = :c WHERE id = :p"),
            {"c": row.review_count + 1, "p": product_id},
        )
    return {"ok": True}
```

1. How many review rows exist afterward, and what is `review_count`?
2. The handler already wraps all of this in a transaction (`engine.begin()`). Why didn't that prevent the problem?
3. Give a fix.

SOLUTION

1. Two review rows (both INSERTs commit), but `review_count` is 6, not 7. Both SELECTs read 5, both compute $5 + 1$, both write 6 -- one increment is lost, and the counter no longer matches the number of reviews.
2. A transaction makes each request's statements commit or roll back together; it does not lock the row the plain `SELECT` read. So both transactions can read `review_count = 5` before either writes, and the read-modify-write races. A transaction is not a lock.
3. Make the increment atomic: `UPDATE products SET review_count = review_count + 1 WHERE id = :p`, which the database does as one locked read-and-write; or lock the row first with `SELECT review_count ... FOR UPDATE` so the second request waits. (Better still, drop the denormalized counter and derive it with `COUNT(*)`.)

Question 3

This endpoint authenticates the caller and even checks account ownership before moving money.

```
class TransferRequest(BaseModel):
    user_id: int
    from_account: int
    to_account: int
    amount_cents: int

@app.post("/api/transfer")
def transfer(body: TransferRequest, user: User = Depends(current_user)):
    with engine.begin() as conn:
        acct = conn.execute(
            text("SELECT owner_id FROM accounts WHERE id = :id"),
            {"id": body.from_account},
        ).first()
        if acct.owner_id != body.user_id:
            raise HTTPException(403, "not your account")
        conn.execute(
            text("UPDATE accounts SET balance = balance - :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.from_account},
        )
        conn.execute(
            text("UPDATE accounts SET balance = balance + :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.to_account},
        )
    return {"ok": True}
```

1. Describe the security hole and give a concrete request that exploits it.
2. Give the fix.
3. There are also multiple correctness problems, separate from the security hole. Find at least one and say how you would address it.

SOLUTION

1. The ownership check compares the account's `owner_id` to `body.user_id`, a value the client supplies, not to the authenticated `user.id`. An attacker sends `{user_id: <victim's id>, from_account: <victim's account>, to_account: <their own>, amount_cents: ...}`; the check passes because they set `user_id` to the victim's, and the transfer drains the victim. Identity must come from the verified token/session, never from the request body.
2. Drop `user_id` from the request and compare against the authenticated identity: `if acct.owner_id != user.id: raise HTTPException(403)`.
3. Any one: no check that `amount_cents > 0` (a negative amount reverses the transfer and steals) -- constrain it in the model (`Field(gt=0)`); no funds check, so the balance can go negative -- guard the debit (`UPDATE ... WHERE id = :id AND balance >= :a` and treat 0 rows affected as a 409); `acct` can be `None` if the account does not exist, which crashes -- 404 first.

Question 4

You are building the React frontend for a shared discussion board (the BBS app from the assignments). One page shows the messages for a board and lets the signed-in user post a new one. Other people post at the same time, so messages from others should appear without the user reloading the page. Assume a parent `<Board>` that renders a `<MessageList>` and a `<NewMessageForm>`. Do not write code -- design it:

1. What pieces of state does this page need, and which component owns each?
2. How do new messages -- the user's own and other people's -- get into the list, and how do you keep it reasonably fresh without a manual refresh? Name the approach and one cost of it.
3. What distinct UI states must the page handle?
4. Name one thing that is easy to get wrong here, and how you would avoid it.

SOLUTION

1. The messages array plus loading and error flags belong to the parent `<Board>` -- it owns the data and passes `messages` down to `<MessageList>`. The in-progress draft text belongs to `<NewMessageForm>` as a controlled input. The board id comes from props/route. Lifting the messages to the parent lets both the list and the post action read and refresh the same data.
2. Load the list on mount with `useEffect` (fetch, then `setMessages`). For the user's own post, POST it and append the server's returned message (or refetch). For other people's messages, poll: refetch on an interval with `useEffect` + `setInterval`, cleared in the effect's cleanup, every few seconds. Cost: polling lags by the interval and adds server load; it is the in-scope option, since real-time push (websockets) was not covered.
3. Loading (first fetch), error (fetch failed), empty (no messages yet), and the loaded list -- plus a disabled / in-flight state on the form while a post is submitting.
4. Any one: showing the loading spinner on every poll blanks the list, so only show it on the first load and not on background refetches; the form double-submitting or not clearing on success; appending the raw input instead of the server's stored message; a slow refetch landing after a newer one (ignore stale results). Pick one and the guard.

Question 5

You are building the ticketing piece of an event app. The app hosts many events, each identified by its own `event_id`. What it has to do:

- Each event has its own fixed pool of free tickets (say 100). Any logged-in user can try to claim a ticket for a given event, and may hold at most one ticket per event (a user can hold tickets to several different events).
- A user should be able to tell whether they currently hold a ticket for a particular event.
- When a popular event's tickets drop, thousands of users hit the app within the same few seconds, and a client on a flaky connection may fire the same request more than once.

Design the API for this. Do not write code, and do not worry about exact status-code numbers. Cover all four:

1. What endpoint(s) does this feature need? For each, give the HTTP method, the URL, and one sentence on what it does and what it returns. (Identity comes from the auth token, not from anything the client sends.)
2. The most important correctness property is that you never hand out more than 100 tickets, even under the stampede. Explain the database-level mechanism that guarantees it, and one sentence on why doing the check in application code (read the count, then insert if there is room) does not.
3. A flaky-connection user fires their claim twice. How do you guarantee they end up with exactly one ticket, not two?
4. One ambiguity in the requirements above that you would settle before building, and the call you would make.

SOLUTION

1. Any clean REST shape, for example: `POST /api/events/{event_id}/claim` -- claims a ticket for the authenticated user if any remain and they do not already hold one, returning their ticket (or telling them it is sold out / they already have one); and `GET /api/events/{event_id}/ticket` -- returns whether the current user holds a ticket for this event. A `DELETE .../ticket` to release one is a reasonable third. Credit is for a sensible shape and taking identity from the token, not for specific URLs.

2. Enforce capacity in the database with an atomic operation: e.g., a `tickets_remaining` counter decremented by `UPDATE ... SET tickets_remaining = tickets_remaining - 1 WHERE id = :e AND tickets_remaining > 0`, treating 0 rows affected as sold out, or an insert guarded by a constraint/row lock. Reading the count and then inserting in app code is a check-then-act race: under the stampede many requests read "room left" before any of them inserts, so they all insert and you oversell.

3. Make the claim idempotent on the user: a `UNIQUE(event_id, user_id)` constraint means a retried claim cannot create a second ticket -- the duplicate insert fails and you return their existing

ticket -- or accept a client idempotency key and dedupe on it. Either way they end with exactly one ticket no matter how many retries arrive.

4. A real ambiguity, e.g.: do over-capacity requests waitlist or just fail; can a user release and re-claim; is the limit one per user, per account, or per device. State the call.

Question 6

Two versions of a component that fetches an item's details when the selected `id` changes. The user clicks quickly through several items. Pick the version you would ship.

```
// Version A
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(`/api/items/${id}`).then(r => r.json()).then(setData);
  }, [id]);
  return <div>{data?.name}</div>;
}

// Version B
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    let active = true;
    fetch(`/api/items/${id}`)
      .then(r => r.json())
      .then(d => { if (active) setData(d); });
    return () => { active = false; };
  }, [id]);
  return <div>{data?.name}</div>;
}
```

1. Justify in three to five sentences, describing the specific bug the other version has and the exact sequence of clicks that triggers it.
2. Both versions still share one shortcoming -- what is it?
3. Version B's cleanup sets a flag rather than aborting the request. Name one thing that approach does NOT do that a real cancellation would.

SOLUTION

1. Ship B. A has the out-of-order race: click item 1, then quickly item 2; if item 1's response arrives after item 2's, A's last `setData` is item 1's and the view shows item 1's data while the prop says item 2. B sets `active = false` in the effect cleanup, which React runs before the next effect, so a superseded response is ignored and only the current id's data is shown.
2. Neither has loading or error states and neither checks `response.ok`, so a slow fetch shows nothing and a failed one shows a stale value with no feedback.
3. The flag only ignores the result; the request still runs to completion and consumes network and server work, and a genuinely slow request cannot be cut off. `AbortController` would cancel the in-flight request itself.

Question 7

Your pipeline: on every push to `main`, GitHub Actions builds a Docker image, pushes it to a registry, and the prod server pulls it and restarts. It works. Reason about three things:

1. The runner that builds the image and the prod server that runs it are different machines. What exactly travels from the runner to the server, through what, and why does the server never need your Dockerfile or your source code?
2. The image build runs the test suite (`RUN pytest`) inside the Dockerfile. Give one reason that is the wrong place for the tests in this pipeline, and where they belong.
3. A database password is needed by the CI job (to run tests against a real database) and by the running container in prod. Where should each come from? They are not the same place.

SOLUTION

1. The built image travels -- not the Dockerfile or the source. The runner runs `docker build` then `docker push` es the image to the registry; the prod server `docker pull` s that same image and runs it, so the registry is the handoff between the two machines. A built image already contains the base Python, the installed dependencies, and the copied app code as layers, so the server runs it as-is and never builds anything -- which is why it needs neither the Dockerfile nor the source.
2. The CI workflow already runs the tests on every push before it builds and pushes, so `RUN pytest` re-runs them on every image build, couples building to test success, slows the build, and bakes test-only files into the image. Tests belong in the CI job (a separate step/job), not in the Dockerfile.
3. The CI secret lives in GitHub Actions Secrets (`${{ secrets.DB_PASSWORD }}`), injected into the workflow run. The prod secret lives in the host/platform's runtime config (env var / dashboard), injected when the container starts. Neither is baked into the image, and CI's secret store is not the prod secret store.